

SMART CLINIC SYSTEM

Domain-Driven Design of Event-Driven Scheduling and Billing Microservices

Architecture and Prototype Implementation Report

Project snapshot reviewed: 3 August 2026

Java 17 | Spring Boot | RabbitMQ | Stripe | PostgreSQL | Docker

Prepared from the uploaded source code. No unimplemented feature is presented as complete.

Executive Summary

The Smart Clinic System is a distributed prototype that separates appointment scheduling from financial processing. The Scheduling Service owns doctor calendars, appointment lifecycle rules, temporal validation, and conflict resolution. The Billing Service owns pricing profiles, proforma invoices, payment ledgers, discount and refund policies, and the Stripe integration. The services exchange appointment lifecycle events through RabbitMQ and maintain separate databases, so neither service reads or writes the other service's data.

The implementation applies Domain-Driven Design through explicit aggregates, value objects, domain services, state transitions, domain events, and a shared language that is reflected in class and method names. Hexagonal Architecture is visible in the separation between application ports and infrastructure adapters. Examples include `AppointmentRepositoryPort` with `AppointmentRepositoryAdapter`, `PaymentGatewayPort` with `ProcessPaymentService`, and `RefundGatewayPort` with `RefundProcessService`.

The core distributed workflow is event-driven. Booking an appointment produces `AppointmentWasBooked`, which causes Billing to issue a draft invoice. Cancelling an appointment produces `AppointmentWasCanceled`, which causes Billing to cancel an unpaid draft or evaluate a paid invoice for refund. An expired invoice produces `InvoiceWasExpired`, which is routed back to Scheduling so that the appointment is cancelled by system automation.

Assessment

The prototype has a coherent domain model and clear service ownership. Its main production-readiness gaps are messaging reliability patterns, idempotency, transaction/event atomicity, and several small implementation inconsistencies identified in Section 3.8.

Report Structure

- 1. Analysis: problem domain, ubiquitous language, invariants, and business flows.
- 2. Architectural Design: bounded contexts, Hexagonal Architecture, persistence ownership, and RabbitMQ communication.
- 3. Implementation: aggregates, policies, application services, Stripe processing, persistence, testing, and prototype constraints.
- 4. Deployment: Docker packaging, Compose topology, startup instructions, Stripe webhook setup, and production/demo timeout profiles.

1. Analysis: The Problem Domain

1.1 Domain Overview

A clinic appointment is not a simple calendar entry. It is constrained by the doctor's effective weekly schedule, temporary unavailability, existing bookings, appointment state, payment deadline, cancellation initiator, and refund policy. The system therefore treats scheduling and billing as related but independent business domains. They coordinate through events instead of sharing tables or calling each other synchronously.

The Scheduling Service answers: **Can this appointment be created or changed, and what is its operational state?** The Billing Service answers: **What amount is due, what financial state is the appointment in, and is a refund allowed?** This division keeps temporal rules out of the billing model and payment-provider concerns out of the scheduling model.

1.2 Core Business Capabilities

Capability	Responsibility
Appointment scheduling	Book, cancel, reschedule, check in, complete, and mark no-show appointments.
Calendar management	Maintain effective weekly schedules and temporary unavailability for each doctor.
Conflict resolution	Reject conflicting bookings and automatically cancel future appointments invalidated by leave or schedule changes.
Pricing	Calculate fees from a doctor pricing profile, discount rules, and a 10% tax rule.
Invoicing	Issue a proforma invoice when an appointment is booked and enforce invoice state transitions.
Payment processing	Create Stripe PaymentIntents, store payment ledgers, verify webhooks, and finalize invoice payment.
Refund processing	Evaluate cancellation eligibility, calculate the refund amount, initiate Stripe refunds, and finalize refund state.
Timeout handling	Release an abandoned checkout or expire an unpaid invoice and cancel the corresponding appointment.

1.3 Ubiquitous Language

Term	Meaning in this project
Appointment	A one-hour consultation reservation between one doctor and one patient.
Active appointment	An appointment in SCHEDULED or CHECKED_IN state. These states may be cancelled or actively rescheduled.
Appointment Calendar	The doctor-specific aggregate that owns effective schedules and unavailability periods.
Effective Schedule	A weekly schedule that becomes valid from a specific date. The latest applicable schedule is used.
Sharp Time	A time aligned to 15-minute increments with no seconds or nanoseconds.
Working Shift	A start and end time during which an appointment must fit completely.
Unavailability	A temporary date-time period during which the doctor cannot accept appointments.
Proforma Invoice	A draft financial document issued after booking. It can be repriced before checkout.
Payment window	The temporary period during which a checkout amount is locked while a Stripe payment is pending.
Expiration date	The final deadline for the invoice. In the current event translation, this is the appointment start time.
Payment ledger	A Payment aggregate entry representing either a deduction or a refund and its gateway transaction ID.
Cancellation Initiator	The actor responsible for cancellation: PATIENT, CLINIC_RECEPTION, or SYSTEM_AUTOMATION.
Refund eligibility	The decision that determines whether cancellation may produce a refund.
Refund strategy	The rule that calculates the approved refund amount after eligibility is established.
Eventual consistency	The interval during which Scheduling and Billing may temporarily represent different stages of the same workflow.

1.4 Domain Invariants

Scheduling invariants

- An appointment date cannot be before the current date.
- Only 60-minute appointments are supported by the Appointment aggregate.
- Start times must use 15-minute increments through SharpTime.
- A booking must fit completely inside the active working shift.
- A booking cannot overlap doctor unavailability or another active future booking.

- Only SCHEDULED appointments can be checked in or marked as no-show.
- Only CHECKED_IN appointments can be completed.
- Only active appointments can be actively rescheduled; only system-cancelled appointments can use the replacement flow.

Billing invariants

- An invoice must have a non-zero base price and a future payment deadline.
- Only a DRAFT invoice can enter PAYMENT_PENDING.
- A successful payment must exactly match the locked invoice amount.
- Only a PAID invoice can enter REFUND_PENDING.
- A confirmed refund must match the approved refund amount.
- A refund ledger can be created only from a successful payment deduction.
- Money cannot be negative and arithmetic requires matching currencies.
- Discounts are capped at 100%, and pricing applies a fixed 10% tax after discount.

1.5 Event Storming and Process Flows

Booking, invoice, and payment flow

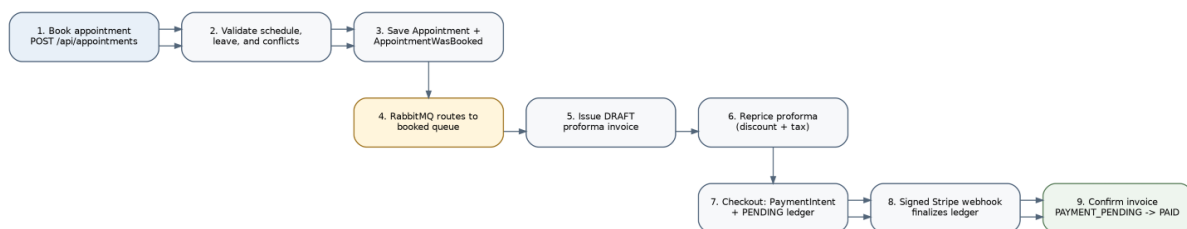


Figure 1. Booking, proforma issuance, checkout, and payment confirmation flow.

The booking transaction ends in the Scheduling Service. Financial processing begins only after Billing receives the booking event. This prevents Stripe or Billing latency from blocking the appointment API. It also means that a newly booked appointment may exist briefly before its invoice is visible.

Unpaid cancellation flow

An unpaid cancellation is represented by a cancelled appointment whose invoice remains in DRAFT. Billing receives AppointmentWasCanceled, loads the invoice by appointment ID, constructs a RefundEvaluationContext, and calls AppointmentInvoice.processCancellation. Because the invoice is still a proforma, the aggregate moves directly to CANCELLED, clears the payment window, and emits no refund request. Stripe is not contacted.

Boundary of the current implementation

The aggregate handles `DRAFT` and `PAID` cancellation paths. A cancellation received while the invoice is `PAYMENT\_PENDING` currently raises an invoice-state exception; this transitional case requires an explicit policy before production use.

Paid cancellation and refund flow

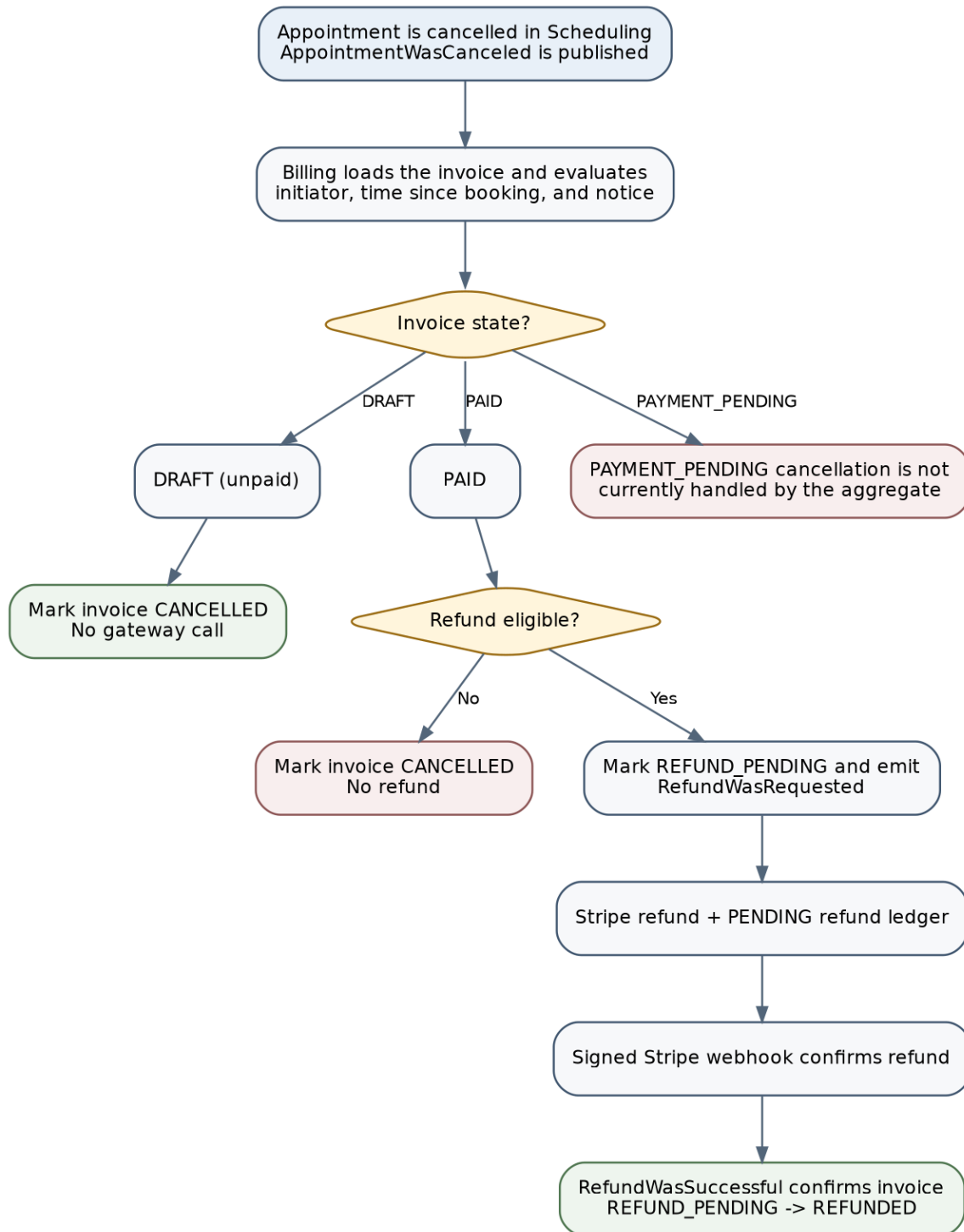


Figure 2. Unpaid cancellation, paid cancellation, and refund decision flow.

For a paid invoice, the refund policy separates **eligibility** from **amount calculation**. Clinic- or system-initiated cancellations force approval. A patient cancellation within 15 minutes of booking also forces approval. Otherwise, a patient cancellation with less than 24 hours' notice is denied. For eligible patient cancellations between 24 and 48 hours before the appointment, the current strategy deducts a fixed 20-unit penalty in the invoice currency; earlier cancellations receive a full refund.

An approved cancellation changes the invoice from PAID to REFUND_PENDING and emits the internal event RefundWasRequested. The refund adapter calls Stripe, creates a pending refund ledger, and waits for a signed webhook. A successful webhook changes the ledger to SUCCESS, emits RefundWasSuccessful, and confirms the invoice as REFUNDED.

Invoice expiration flow

The Billing scheduler runs every five seconds and scans DRAFT and PAYMENT_PENDING invoices. If the final invoice deadline has passed, the invoice becomes EXPIRED and emits InvoiceWasExpired. Billing publishes this event through RabbitMQ. Scheduling consumes it and cancels the appointment with SYSTEM_AUTOMATION, which in turn publishes the normal appointment cancellation event. The resulting duplicate financial cancellation is harmless for an already expired invoice because processCancellation returns without another transition.

2. Architectural Design

2.1 Bounded Contexts

The project defines two bounded contexts with different models, terminology, and consistency requirements. The same appointment identifier crosses the boundary, but the aggregates do not. Scheduling owns operational truth; Billing owns financial truth.

Bounded context	Owned model and decisions	Database
Scheduling	Appointment, AppointmentCalendar, schedules, shifts, unavailability, temporal conflicts, check-in, completion, no-show, cancellation, and rescheduling.	scheduling_db
Billing	AppointmentInvoice, Payment, PricingProfile, pricing policies, refund policies, Stripe transactions, reconciliation, and timeout processing.	billing_db

The split is justified because calendar availability and financial reconciliation change for different reasons. A new refund rule should not require changes to appointment conflict detection. A new scheduling constraint should not require changes to Stripe adapters. Independent databases also prevent accidental cross-context joins and preserve ownership.

2.2 System Architecture

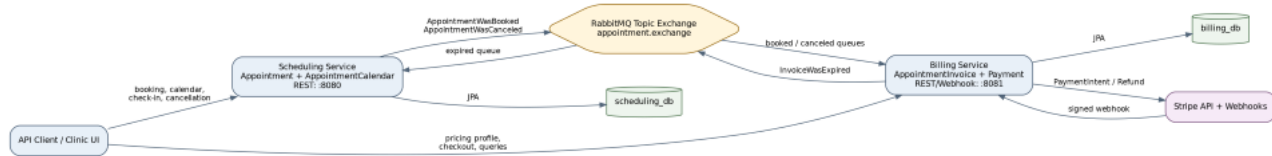


Figure 3. Runtime architecture and event directions.

External clients use REST endpoints for commands and queries. Cross-service communication is exclusively asynchronous in the current prototype. Stripe is the only external synchronous dependency used during payment or refund initiation; transaction completion is webhook-driven.

2.3 Hexagonal Architecture

Both services place business behavior behind inbound use-case interfaces and outbound ports. Infrastructure code depends on the ports; the core domain does not depend on Spring MVC, RabbitMQ, JPA, or Stripe.

Role	Examples from the codebase
Inbound port	SchedulingUseCase, CalendarUseCase, ProformaUseCase, InitiateCheckoutUseCase, InitiateCancellationUseCase, FinalizeTransactionUseCase
Inbound adapter	REST controllers, RabbitMQ listeners, Stripe webhook controller, and scheduled timeout task.
Outbound port	AppointmentRepositoryPort, InvoiceRepositoryPort, PaymentRepositoryPort, PaymentGatewayPort, RefundGatewayPort, DomainEventPublisher
Outbound adapter	JPA repository adapters, RabbitMQ publishers, ProcessPaymentService, and RefundProcessService.

The Stripe dependency is isolated particularly well. InitiateCheckoutService depends on PaymentGatewayPort, not the Stripe SDK. ProcessPaymentService translates domain Money into Stripe minor units and returns a provider-neutral PaymentIntentResponse. Refund initiation follows the same pattern through RefundGatewayPort.

2.4 Asynchronous Communication with RabbitMQ

A durable topic exchange named appointment.exchange routes three integration-event categories. Each service declares only the queues it consumes. This creates temporal decoupling: the producer completes without waiting for the consumer's database or external gateway.

Event	Producer	Consumer	Routing key	Queue	Purpose
AppointmentWasBooked	Scheduling	Billing	appointment.booked.key	appointment.booked.queue	Issue a draft invoice.

Event	Producer	Consumer	Routing key	Queue	Purpose
AppointmentWasCanceled	Scheduling	Billing	appointment.cancelled.key	appointment.canceled.queue	Cancel a draft or evaluate refund.
InvoiceWasExpired	Billing	Scheduling	appointment.expired.key	appointment.expired.queue	Cancel the unpaid appointment by automation.

Why RabbitMQ instead of synchronous REST

- Appointment commands do not wait for invoice creation or Stripe availability.
- The services can be deployed and scaled independently.
- The event log expresses business facts rather than remote procedure calls.
- Temporary consumer downtime does not require the producer to know the consumer's endpoint.
- The architecture exposes eventual consistency explicitly instead of hiding a distributed transaction behind chained REST calls.

Consistency and reliability implications

The design intentionally avoids a distributed ACID transaction. A database commit and an event publication are separate operations. The current code publishes after repository save inside the application transaction, but it does not use a transactional outbox. Therefore, a process failure between the database operation and message publication can leave the contexts inconsistent. Duplicate delivery is also possible in broker-based systems, while the current consumers do not implement explicit idempotency keys.

Production hardening priority

Add a transactional outbox in each producer, idempotent consumers keyed by event or aggregate ID, publisher confirms, retry policy, and dead-letter queues. These mechanisms preserve the existing domain model while strengthening delivery guarantees.

2.5 CQRS-Style Read Separation

Command controllers call use cases and aggregates. Query controllers call dedicated DAOs such as `AppointmentQueryDao`, `InvoiceQueryDao`, and `PaymentQueryDao`. This is a lightweight CQRS pattern: the write model protects invariants, while read endpoints return purpose-specific projections without reconstructing aggregates.

3. Implementation: The Prototype

3.1 Technology Stack

Area	Implementation
Language	Java 17
Application framework	Spring Boot 4.0.6 in Scheduling; Spring Boot 4.1.0 in Billing
Web/API	Spring Web MVC and Jakarta Validation
Persistence	Spring Data JPA / Hibernate with PostgreSQL
Messaging	Spring AMQP with RabbitMQ topic exchange and JSON conversion
Payments	Stripe Java SDK 33.1.0 and Stripe CLI webhook forwarding
Build	Maven Wrapper and multi-stage Docker builds
Containers	Docker Compose, PostgreSQL 15 Alpine, RabbitMQ 3 Management Alpine
Testing	JUnit-based domain tests; H2 is available to Billing tests

3.2 Scheduling Domain Model

Appointment aggregate

Appointment owns identity, patient, doctor, one-hour time slot, current status, cancellation initiator, creation/update timestamps, and an optional link to the appointment from which it was rescheduled. State-changing methods enforce lifecycle rules and create domain events only when a business fact must cross the service boundary.

State	Permitted meaning
SCHEDULED	Initial state; may be checked in, cancelled, marked no-show, or rescheduled.
CHECKED_IN	May be completed, cancelled, or actively rescheduled.
COMPLETED	Terminal operational state.
CANCELLED	Terminal state containing the cancellation initiator.
NO_SHOW	Terminal state for a scheduled patient who did not attend.

AppointmentCalendar aggregate

AppointmentCalendar selects the most recent EffectiveSchedule applicable to the requested date, checks whether the doctor is working, checks temporary unavailability, and compares the requested slot with already booked slots. The AppointmentConflictDomainService handles rules that require both a calendar change and a collection of appointments. When new leave or a new schedule invalidates bookings, CalendarService cancels the conflicts with SYSTEM_AUTOMATION and publishes cancellation events.

3.3 Billing Domain Model

AppointmentInvoice aggregate

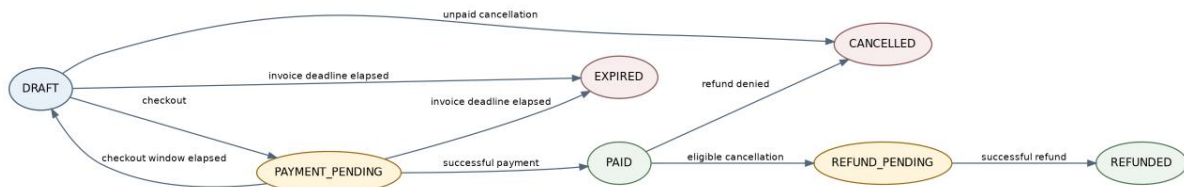


Figure 4. Implemented AppointmentInvoice state transitions.

AppointmentInvoice is the financial process manager for one appointment. It stores base price, discount, total amount, approved refund amount, final invoice deadline, checkout window, timestamps, and invoice status. The aggregate prevents invalid transitions rather than allowing application services to set status fields directly.

Payment aggregate

Payment is an immutable-identity ledger entry with mutable status. It distinguishes PAYMENT_DEDUCTION from REFUND. Both start as PENDING; Stripe results move them to SUCCESS or FAILED, while a checkout

timeout can move a pending payment to **EXPIRED**. Successful deductions emit **PaymentWasSuccessful**; successful refunds emit **RefundWasSuccessful**.

PricingProfile and pricing policy

Each doctor has a **PricingProfile** containing an hourly rate and currency. Appointment duration is converted from minutes to hours when the booking event enters **Billing**. The pricing policy evaluates stackable and exclusive discounts, selects the greater result, then applies 10% tax.

Rule	Type	Discount	Condition
Immediate settlement	Exclusive	10%	Within 15 minutes of proforma creation.
Prompt payment	Stackable	2%	Within 120 minutes of proforma creation.
Early clearance	Stackable	3%	At least three days before the payment deadline.

Example: within the first 15 minutes, the exclusive 10% discount is greater than the possible 5% stacked discount, so 10% is selected. After the immediate window, prompt payment and early clearance may combine to 5%.

Refund policy

Cancellation case	Eligibility	Amount strategy
Clinic reception or system automation	Force approve	Full refund.
Patient; less than 15 minutes since booking	Force approve	Full refund.
Patient; less than 24 hours before appointment	Deny	No refund; invoice becomes cancelled.
Patient; 24-48 hours before appointment	Approve	Total paid minus 20 units in the same currency.
Patient; more than 48 hours before appointment	Approve	Full refund.

3.4 Application Services and Orchestration

Service	Responsibility
SchedulingService	Validates availability, changes Appointment, persists it, and publishes booking/cancellation events.
CalendarService	Changes schedules/unavailability, detects conflicts, cancels affected appointments, and publishes events.
ProformaService	Creates invoices from booking events and recalculates price before checkout.
InitiateCheckoutService	Locks the invoice amount, creates a Stripe PaymentIntent, and stores a pending payment ledger.
FinalizeTransactionService	Applies Stripe results to ledgers and propagates successful results to the invoice.
InitiateCancellationService	Builds refund context, evaluates policy, changes invoice state, and publishes internal refund requests.
RefundInitiationService	Calls Stripe for a refund and records a pending refund ledger.
TimeOutService	Releases expired checkout windows or expires invoices and publishes appointment-expiration events.

3.5 Persistence and Data Ownership

Domain aggregates are mapped to JPA entities through explicit mapper classes. Repository adapters catch Spring data-access exceptions and translate them into application-specific infrastructure exceptions. Scheduling stores appointments and doctor-calendar collections. Billing stores invoices, payment ledgers, and doctor pricing profiles. Docker initializes two databases on one PostgreSQL server, preserving logical ownership while keeping local deployment simple.

The project also separates write and read access: aggregate repositories reconstruct domain objects, while query DAOs return response projections directly. This avoids exposing JPA entities through the API.

3.6 Stripe Integration

- **ProcessPaymentService** creates a card **PaymentIntent** using amount in minor currency units.
- **RefundProcessService** creates a Stripe refund against the original **PaymentIntent** ID.
- **StripeSignatureVerifier** validates every webhook with the configured endpoint secret.
- **StripeTranslatorDispatcher** handles payment success/failure and refund-update events, ignoring unrelated event types.

- Webhook translators return provider-neutral `TransactionResultCommand` objects to the application layer.
- The invoice is not marked paid or refunded when the API request is sent; it changes only after a verified webhook confirms the result.

3.7 Verification and Test Design

The source tree contains 14 Scheduling test classes and 31 Billing test classes. The tests concentrate on domain behavior: identifiers, time values, weekly schedules, overlap semantics, invoice transitions, payment transitions, money and percentage arithmetic, pricing rules, refund eligibility, and refund strategies. This emphasis is appropriate for DDD because the highest-value tests verify invariants without requiring Spring or external infrastructure.

The uploaded build artifacts contain older Scheduling Surefire reports for 54 passing tests across appointment and temporal classes. They do not represent a complete fresh build of the uploaded source. Before submission, the full Maven test suites should be run from a clean checkout, followed by integration tests for PostgreSQL, RabbitMQ event contracts, duplicate delivery, Stripe webhook signatures, and timeout behavior.

3.8 Prototype Constraints

Priority	Area	Current condition	Required action
High	Messaging atomicity	Repository save and RabbitMQ publication are not protected by a transactional outbox.	Add outbox tables, relay workers, publisher confirms, and idempotent consumers.
High	Duplicate/error handling	No explicit message ID, deduplication store, retry policy, or dead-letter queue is configured.	Add event IDs, consumer inbox/deduplication, bounded retry, and DLQ routing.
High	System-cancelled replacement	<code>rescheduleSystemCanceledAppointment</code> creates a new appointment with a booking event but publishes events from the old appointment.	Publish and clear the new appointment's domain events so Billing issues its invoice.
Medium	Optimistic locking	<code>SchedulingService</code> catches optimistic-lock exceptions, but <code>AppointmentEntity</code> has no <code>@Version</code> field.	Add a version column or remove the unsupported concurrency claim.

4. Deployment

4.1 Containerization

Both services use multi-stage Dockerfiles. Maven and Eclipse Temurin 17 compile the Spring Boot JAR in the builder stage. The runtime stage uses the smaller Eclipse Temurin 17 JRE image and starts the application with `java -jar app.jar`. This keeps build tooling out of the final image and improves repeatability.

4.2 Docker Compose Topology

Service	Image/build	Host ports	Purpose
rabbitmq	rabbitmq:3-management-alpine	5672, 15672	Event broker and management UI; health-checked.
postgres	postgres:15-alpine	5432	Hosts scheduling_db and billing_db; persistent volume.
scheduling-service	Local multi-stage build	8080	Scheduling API and appointment-expiration consumer.
billing-service	Local multi-stage build	8081	Billing API, RabbitMQ consumers, scheduler, and Stripe webhook.
stripe-cli	stripe/stripe-cli:latest	None	Forwards Stripe events to Billing inside the Compose network.

4.3 Prerequisites and Environment

Docker and Docker Compose are required. The root `.env` file must provide Stripe test credentials. Secrets must not be committed, submitted, or embedded in the report.

```
STRIPE_API_KEY=sk_test_...
STRIPE_PAYMENT_WEBHOOK_SECRET=whsec_...
```

4.4 Running the System

From the project root:

```
docker compose up --build
```

For Docker Compose v1 installations, the equivalent command is:

```
docker-compose up --build
```

To start in detached mode and inspect logs:

```
docker compose up --build -d
docker compose ps
docker compose logs -f scheduling-service billing-service stripe-cli
```

To stop the system while preserving PostgreSQL data:

```
docker compose down
```

To stop the system and remove the database volume:

```
docker compose down -v
```

4.5 Stripe CLI Webhook Secret

The Stripe CLI prints a webhook signing secret when it starts listening. Billing must use the secret for the active listener session. A reliable local procedure is:

1. Start RabbitMQ, PostgreSQL, Scheduling, and Billing.
2. Start the Stripe CLI and read its log output.
3. Copy the displayed ``whsec_...`` value into ``STRIPE_PAYMENT_WEBHOOK_SECRET``.
4. Recreate or restart the Billing container so the new secret is loaded.
5. Trigger a Stripe test event and verify that Billing returns HTTP 200 for the signed webhook.

```
docker compose up -d rabbitmq postgres scheduling-service billing-service
docker compose up -d stripe-cli
docker compose logs -f stripe-cli
```

```
# update .env with the displayed whsec_... value
docker compose up -d --force-recreate billing-service
```

4.6 Production and Demo Timeout Profiles

The checkout timeout is injected into `InitiateCheckoutService` through the property `clinic.checkout.payment-timeout`. The checked-in `application.properties` currently sets it to **20 seconds**, and the timeout scheduler checks every five seconds. This is suitable for a demonstration but not for production.

Recommended production configuration:

```
services:
  billing-service:
    environment:
      - CLINIC_CHECKOUT_PAYMENT_TIMEOUT=2h
```

Recommended demonstration configuration:

```
services:
  billing-service:
    environment:
      - CLINIC_CHECKOUT_PAYMENT_TIMEOUT=15s
```

The environment variable overrides the packaged property through Spring Boot's external configuration precedence. The report should therefore state: **The production checkout grace period is configured as two hours; the Docker Compose demo profile overrides it to 15 seconds so timeout behavior can be observed immediately.** To make that statement true, the Compose environment value and source default must be aligned before submission.

4.7 Demonstration Scenario

1. Create a doctor pricing profile in Billing.
2. Create an effective weekly schedule for that doctor in Scheduling.
3. Book a one-hour appointment on a valid 15-minute boundary.
4. Query Billing for the draft invoice created from the booking event.
5. Update the proforma to apply pricing policies, then initiate checkout.
6. Complete a Stripe test payment and observe the invoice move to `PAID` after the webhook.
7. Cancel the appointment and observe either cancellation or refund processing according to initiator and timing.
8. Repeat without payment and allow the invoice to expire; observe the event-driven system cancellation.

Conclusion

The Smart Clinic System demonstrates a strong prototype-level application of DDD to a distributed workflow. The bounded contexts are meaningful, the aggregates protect business state, the policy objects make financial rules explicit, and the ports-and-adapters structure isolates databases, RabbitMQ, and Stripe. The event-driven design correctly exposes eventual consistency and prevents financial processing from blocking scheduling commands.

The next engineering step is not to add more domain features, but to harden delivery and failure behavior: transactional outbox, idempotent consumers, dead-letter handling, integration tests, and correction of the small inconsistencies identified in Section 3.8. With those changes, the architecture can evolve from a convincing academic prototype into a robust distributed clinic platform.

Appendix A. Public API Summary

Service	Method	Endpoint	Purpose
Scheduling	POST	/api/appointments	Book an appointment.
Scheduling	POST	/api/appointments/{id}/cancel	Cancel an appointment.
Scheduling	POST	/api/appointments/{id}/checkin	Check in the patient.
Scheduling	POST	/api/appointments/{id}/complete	Complete a checked-in appointment.
Scheduling	POST	/api/appointments/{id}/noshow	Mark a scheduled appointment as no-show.
Scheduling	POST	/api/appointments/{id}/reschedule/active	Cancel and replace an active appointment.
Scheduling	POST	/api/appointments/{id}/reschedule/system-cancelled	Replace a system-cancelled appointment.
Scheduling	POST	/api/calendars/{doctorId}/unavailabilities	Add doctor unavailability and cancel conflicts.
Scheduling	POST	/api/calendars/{doctorId}/schedules	Add an effective weekly schedule and cancel conflicts.
Billing	POST	/api/profile/create	Create a doctor pricing profile.
Billing	POST	/api/profile/update	Intended to update a pricing profile; currently wired to create.
Billing	POST	/api/payments/{invoiceId}/update/proforma	Recalculate discount and tax.
Billing	POST	/api/payments/{invoiceId}/initiate/payment	Create PaymentIntent and pending ledger.
Billing	POST	/api/stripe/webhook/payments	Receive signed Stripe payment/refund events.
Both	GET	/api/queries/...	Read appointment, invoice, payment, and reporting projections.

Appendix B. Source-Code Evidence Map

Report topic	Primary implementation location
Scheduling aggregate	domain/Appointment.java
Calendar aggregate	domain/AppointmentCalendar.java
Conflict domain service	domain/service/AppointmentConflictDomainService.java
Scheduling orchestration	application/service/SchedulingService.java, CalendarService.java
Scheduling events	domain/events/AppointmentWasBooked.java, AppointmentWasCanceled.java
Billing invoice aggregate	domain/invoice/AppointmentInvoice.java
Payment ledger aggregate	domain/payment/Payment.java
Pricing policy	domain/policy/pricing/...
Refund policy	domain/policy/refund/...
Stripe adapters	infrastructure/gateway/stripe/...
RabbitMQ adapters/configuration	infrastructure/.../messaging/..., RabbitConfig.java
Container deployment	Dockerfile, root docker-compose.yml, postgres-init/init.sql